



New York University
Computer Science Department
Introduction to Software Development

Dr. Nader Nassar

Homework 7

Deadline: *See NYUclasses for the deadline, 5% off per day after the deadline (3 days maximum).*

Please read the guidelines carefully to avoid receiving a zero on the HW:

- Please use the following naming convention “ExerciseX.java”, where X is the number of the exercise.
- Apply this convention to all future homework assignments during the semester.
- Compile and run the program for each exercises.
- Make an archive (zip file or compressed file) with all the java files (the .java files NOT the .class files) and post it on NYU Classes (StudentName_Homework_X.zip)
- It is your responsibility to make sure that the Zip files have your actual files. You may send the file to yourself by email to double check before you upload on NYU classes.
- If the graders cannot open the file, you will receive a grade of zero.
- If you send the .class files instead of the .java files (source files) you will receive a zero.
- Cheating will be severely addressed with an immediate zero on the homework and a report to the academic advisor and the administration.
- You will automatically lose 50% of the points for an exercise if the program does not compile and run correctly

Exercise. 1 (20pts) Design The Account Class.

Design a class named Account that contains:

1. A private int data field named id for the account (default value 0)
2. A private double data field named balance for the account (default value 0)
3. A private double data field named annualInterstRate that stores the current interest rate (default 0). Assume all account have the same interest rate.
4. A private Date data field named dateCreated that stores the date when the account was created.
5. A default constructor that creates the default account.
6. A constructor that that creates an account with the specified id and initial balance.
7. The accessor and mutator methods for id, balance and annualInterstRate
8. The accessor method for dateCreated
9. A method named getMonthlyInteraset() that returns the monthly interest
10. A method named withdraw that withdraws a specified amount from the account.
11. A method named deposit that deposits a specified amount to the account.

Write a test program that creates an Account object with an account ID of 1122, a balance of \$20,000 and an annual interest of 4.5%. Use the withdraw method to withdraw \$2,500, use the deposit method to deposit \$3,000, and print the balance, the monthly interest and the date with this account was created.

Exercise. 2 (30pts)

- Define a class named **PhoneBookEntry**. Each PhoneBookEntry has an:
 - **Id** (int),
 - **first name** (String),
 - **last name** (String),
 - **email** (String),
 - **zip code** (String), and
 - **a phone number** (String).
- Examples of Setters and Getters with signatures like:
 - **public void setFirstName(String FName) { ... }**
 - **public String getFirstName() { ... }**

You will need to define the setters and getters for all private class members variables
- You need to create the following *constructors (you might need other constructors)*:
 - You will need to create a default constructor
 - **public PhoneBookEntry() { ... }**
 - A Constructor that takes as arguments all attributes
 - A Constructor that accepts only **firstName** and **phoneNumber** as arguments
- Define a method **printBookEntry()** that prints all the data fields (first name, last name, zip code, phone number etc.). This completes the definition of the class **PhoneBookEntry**.
- Next, you will need create a class called **PhoneBook** that instantiates three objects of type **PhoneBookEntry**. The attributes of each object are inputs from the user. This class will have the main method.

(A) You will need to create objects with with the appropriate constructors accordingly and object attributes are as follows:

First Object: 5,John,Smith,jsmith@gmail.edu,20037,2023334444

Second Object: James,202344344

Third Object: Sarah

- Provide the code for the following in the main method of Phonebook – **You must comment your code.**

(B) Demonstrate how you can use getters and setters to change the first name and phone number of attributes of objects in (A):

1. Change the home number of John Smith to 202555555
2. Print the attributes of John Smith using the method **PrintBookEntry()**.
3. Assign the Zipcode of John Smith to James's.

Create a folder for this exercise and include all your classes in the same folder. Name this folder Exercise 2

Exercise. 3 (50pts)

An Array of PhoneBookEntries

- This part uses the PhoneBookEntry defined in Exercise#2
- The objective of this exercise is to create a phone book application using the object-oriented programming paradigm.
- Create a class named **PhoneBookDirectory**. The PhoneBookDirectory has an array of 6 PhoneBookEntry' objects (for simplicity use size 6 so you can test your program).
- Define in **PhoneBookDirectory** and use methods that allow to perform the following:

1. Add an entry to the phonebook

Adding an entry to the phone to the array in first empty space in the array.
The method signature is as follows: (You must use this signature)

```
public int addEntry(PhoneBookEntry entry) { ... }
```

the method returns 1 if the object entry was successfully added or 0 otherwise (in case the array is full).

2. Print all phonebook entries. This may look like:

```
public void printAllEntries() { ... }
```

3. Search for an entry by Phone Number (Linear Search):

```
public int LinearSearchByPhoneNumber(String PhoneNumber) { ... }
```

the method returns 1 if phone number is found found and 0 otherwise.

4. Search for an entry by id (Using Binary Search):

```
public PhoneBookEntry SearchByIdBinarySearch(int id) { ... }
```

returns a PhoneBookEntry if found and an empty PhoneBookEntry if NOT found.

5 Edit an entry (allows to edit any attributes (data fields) they want of a specific phone book entry of a given last name and firstname)

```
public int Edit(String firstName, String lastName) { ... }
```

return a 1 if the object was found and edited and returns 0 if the entry of the given userName and lastName do not exist.

Create a folder for this exercise and include all your classes in the same folder. Name this folder Exercise 3